

Quiz

QuickSort mit zufälligem Pivot Element



- Las-Vegas Algorithmus mit erwarteter Laufzeit $O(n \ln n)$

Beweis: Sei $(a_1, a_2, \dots, a_n)$ das Array nach dem Sortieren

Seien a_i und a_j zwei beliebige Elemente, wobei $i < j$

Sei $I_{i,j} = \begin{cases} 1 & \text{falls } a_i \text{ mit } a_j \text{ verglichen wird} \\ 0 & \text{sonst} \end{cases}$ eine Indikatorvariable

Irgendwann wird das erste mal ein Pivot $p \in \{a_i, \dots, a_j\}$ ausgewählt.

Case $p \in \{a_i, a_j\}$: a_i und a_j werden einmal verglichen

Case $p \in \{a_{i+1}, \dots, a_{j-1}\}$: a_i und a_j werden nie verglichen, weil sie a_i im linken rekursiven Aufruf ist und a_j im rechten

$(a_1, \dots, a_{i-1}, \boxed{a_i, a_{i+1}, \dots, a_{j-1}, a_j}, a_{j+1}, \dots, a_n)$
 $j-i+1$ viele Elemente

$$\mathbb{E}[I_{i,j}] = \Pr[a_i \text{ wird mit } a_j \text{ verglichen}] = \frac{2}{j-i+1}$$

$$\mathbb{E}[\text{totale Anzahl Vergleiche}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n-1} \mathbb{E}[I_{i,j}]$$

Summe über alle möglichen Paare $1 \leq i < j \leq n$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n-1} \frac{2}{j-i+1}$$

$$\mathbb{E}[I_{i,j}] = \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k}$$

Index Substitution $k = j-i+1$

$$j=i+1 \Leftrightarrow j-i+1=2 \Leftrightarrow k=2$$

$$j=n-1 \Leftrightarrow j-i+1=n-i+1 \Leftrightarrow k=n-i+1$$

$$\leq \sum_{i=1}^n \sum_{k=1}^n \frac{2}{k}$$

Summen gehen bis n

3

$$= 2 \cdot \sum_{i=1}^n \sum_{k=1}^n \frac{1}{k}$$

Distributivgesetz

*

$$= 2n \cdot H_n$$

Harmonische Reihe $H_n \stackrel{\text{def}}{=} \sum_{k=1}^n \frac{1}{k}$

$$\leq \underline{\underline{O(n \log n)}}$$

weil $H_n \leq \ln(n) + O(1)$

QuickSelect: Es gibt eine deterministische Version von QuickSort mit Laufzeit $O(n \log n)$

Duplikate finden mit direktem Sortieren

Universum: U ist die Menge aller möglichen Elementen

Datensatz: $D = (s_1, s_2, \dots, s_n)$ ist eine Folge von n Elementen aus einem Universum U

Duplikat: Ein Paar (i, j) sodass $s_i = s_j$, wobei $i < j$

1.) Sortieren

2.) Liste durchlaufen und dabei alle Duplikate ausgeben

Bsp $(4, 19, 1, 2, 4, 5, 2, 4)$) Sortieren
 Index: $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{matrix}$
 $(1, \underline{2}, \underline{2}, \underline{4}, \underline{4}, \underline{4}, 5, 19)$

Duplikate: $(2, 3), (4, 5), (4, 6), (5, 6)$

Laufzeit: $O(\underbrace{n \log n}_{\text{Sortieren}} + \underbrace{|\text{Duplikate}|}_{\text{Duplikate ausgeben}})$

Wie viele Duplikate gibt es höchstens bei n Elementen? $\rightarrow \sum_{i=1}^{n-1} 1 = \frac{n(n-1)}{2}$

Duplikate finden mit Hashfunktion

Motivation: Vergleiche von "schwierigen" Elementen (z.B. Graphen, DNA-Strings) können teuer sein

Hashfunktion: $h: U \rightarrow \{1, \dots, m\}$ berechnet für ein Element $u \in U$ den Hashwert $h(u)$

- m soll viel kleiner als $|U|$ sein
- h soll effizient berechenbar sein
- h soll zufällig gleichverteilt sein. $\forall u \in U \forall i \in [m]: \Pr[h(u) = i] = \frac{1}{m}$
- h hat die Eigenschaft: $s_i = s_j \Rightarrow h(s_i) = h(s_j)$

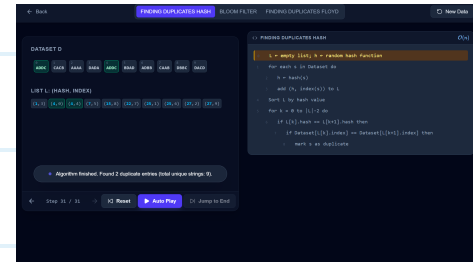
Kollisionen: sind Paare (i, j) mit $s_i \neq s_j$ aber $h(s_i) = h(s_j)$

1.) Hashen und Index merken

2.) Sortieren nach Hashwert aufsteigend

3.) Liste durchlaufen und dabei alle Hashwert-Duplikate

überprüfen und ausgeben, wenn es keine Kollisionen sind



Laufzeit: $O(n \log n + |\text{Duplikate}| + |\text{Kollisionen}|)$

Hashen und Sortieren Duplikate ausgeben Kollisionen überprüfen

Für $m = n^2$: ist die erwartete Anzahl Kollisionen konstant

ist der zusätzliche Speicher in $O(n \log n)$

könnt diese Theorie für die Prüfung

Wiederholungen finden mit Bloom Filter

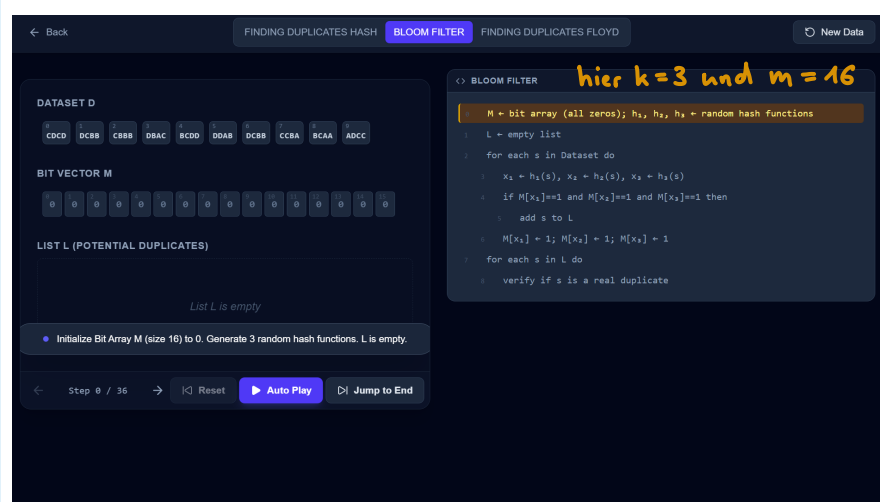
Wiederholung: ein Index $j \in [n]$, sodass $s_i = s_j$ für ein $i \in \{1, \dots, j-1\}$

Hashfunktionen: $\underbrace{h_1: U \rightarrow [m], h_2: U \rightarrow [m], \dots, h_k: U \rightarrow [m]}_{k \text{ viele Hashfunktionen}}$, alle zufällig gleichverteilt

Hash-Vektor von s_i : $\underbrace{(h_1(s_i), h_2(s_i), \dots, h_k(s_i))}_{\text{alle } k \text{ Hashfunktionen auf } s_i \text{ angewendet}}$

M : ist ein boolean Array der Länge m , anfangs auf $M = (0, \dots, 0)$ initialisiert

Algorithmus:

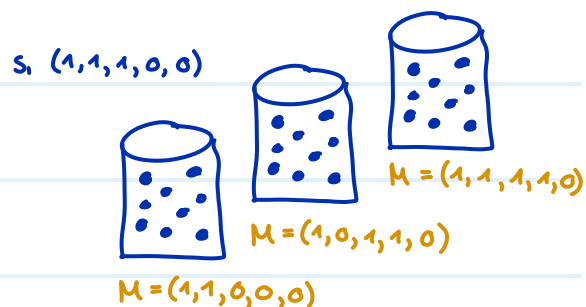


falscher L -Eintrag: falls $s_i \in L$ aber $s_i \notin \{s_1, s_2, \dots, s_{i-1}\}$

Für $k = \ln(n)$ und $m = n \ln(n)$: Anzahl falsche L -Einträge ist konstant

Laufzeit: $O(kn)$
 n mal k Hashwerte berechnen

Anwendung Router, Datenbanken



nur sehr wenig dazu an der Prüfung